

Les vues et les séquences

II1

2024/2025

hela.boukef@ensi-uma.tn

Les vues_(1/6)

Définition

Une vue est une table virtuelle constituée par des sous-ensembles de données d'une ou plusieurs tables. Elle est définie par une requête prédéfinie stockée dans la base de données. Elle est exploitée comme une table mais elle **ne nécessite pas d'espace physique de stockage.**

Les vues(2/6)

Utilités

- Pour des raisons de sécurité : restreindre l'accès aux données.
- Conserver des données de synthèse afin d'éviter les requêtes très complexes.
- Permettre une certaine indépendance des données.
- Présenter des différentes vues pour les mêmes données...

Les vues(3/6)

Création d'une vue :

Cas d'une vue simple (à partir d'une table).

```
CREATE VIEW Nom_Vue AS  
  SELECT * | COLONNE1 [,COLONNE2...]  
FROM Nom-Table  
WHERE Conditions;
```

Exemple: Créer une vue des noms, postes et salaires des employés du service n°10.

RQ! On peut renommer les noms des colonnes dans la vue en utilisant les alias dans la sous-requête.

Les vues(4/6)

Remarques:

- On peut décrire et faire des sélections dans les vues comme les tables
(→ on peut appliquer le langage d'interrogation des données sur les vues)

*SELECT * FROM Nom_Vue;*

Suppression d'une vue:

La suppression d'une vue s'effectue de la même façon qu'une table.

Drop View Nom_Vue;

Les vues(5/6)

Cas d'une vue complexe (à partir de plusieurs tables)

```
CREATE VIEW Nom_Vue AS
SELECT * | COLONNE1 [, COLONNE2,...]
FROM TABLE1, TABLE2
WHERE Conditions
[GROUP BY...] ;
```

RQ! Les vues complexes sont souvent utilisées pour la synthèse des données.

Exemple : Créer une vue qui contient, pour chaque service, le nom du service, le nombre d'employés le salaire moyen dans ce service.

Les vues(6/6)

L'option WITH CHECK OPTION

L'option WITH CHECK OPTION est une option de l'instruction CREATE qui peut être ajoutée afin que toutes les instructions UPDATE et INSERT répondent aux conditions de la définition de la vue.

Exemple :

```
CREATE VIEW Pers_Embauche AS  
SELECT NumP, NomP, PosteP, datembp  
FROM personne  
WHERE datembp IS NOT NULL  
With check option;
```

Dans ce cas, l'option WITH CHECK OPTION *refuse toute valeur NULL dans la colonne « datembp »!*

Exercice

1. Créer une vue emp_sec qui représente la liste des employés occupant un poste de secrétaire.
2. Utiliser la vue emp_sec pour insérer les deux enregistrements suivants dans les tables sous-jacentes :
 - (8001,'ZAYANI','INGENIEUR','11-03-15',900,60)
 - (8005,'TLILI','SECRETAIRE','03-09-04',450,10)
3. Consulter la vue emp_sec et les tables sous-jacentes. Qu'est-ce que vous remarquez ?
4. Refaire le même travail que les questions 1,2 et 3 en ajoutant la clause WITH CHECK OPTION à la définition de la vue. Apporter les modifications nécessaires.
5. A partir de la vue précédemment créée, définir une troisième vue emp300 qui donne les personnes qui touchent plus que 300.
6. Utiliser la vue emp300 pour insérer la ligne suivante :
 - (8010,'DAHMANI','COMMERCIAL','23-06-11',250,30)
 - Qu'est-ce que vous constatez ? Justifier cette situation

Les séquences (1/10)

Définition

Une séquence est un objet de base de données, au même titre qu'une table, une vue ou un index, dont le rôle est de générer automatiquement une série de numéros (entiers) uniques. Elle est généralement utilisée pour générer les valeurs des clés primaires et elle peut être partagée par plusieurs utilisateurs.

Les séquences (2/10)

Syntaxe

```
CREATE SEQUENCE <nom-séquence>  
[INCREMENT BY <valeur-pas>]           (par défaut: 1)  
[START WITH <valeur-début>]  
[MAXVALUE <valeur-max> | NOMAXVALUE]  
(par défaut: NOMAXVALUE)  
[MINVALUE <valeur-min> | NOMINVALUE]  
[CYCLE | NOCYCLE]                     (par défaut: NOCYCLE)  
[CACHE <nb-valeurs>| NOCACHE];  
( par défaut: NOCACHE)
```

Les séquences (3/10)

INCREMENT BY :spécifie l'intervalle d'incrémentation. Cette valeur peut être positive ou négative mais ne peut pas valoir 0. Par défaut, l'incrémentation vaut 1.

START WITH :spécifie la première valeur délivrée par la séquence. Par défaut, cette valeur est la valeur minimum pour les incrémentations positives et la valeur maximum pour les négatives.

MINVALUE :spécifie la valeur minimum de la séquence. Doit être au minimum égal à la valeur spécifiée par START WITH.

MAXVALUE :spécifie la valeur maximum que le compteur peut générer.

Les séquences (4/10)

CYCLE : permet à la séquence de recommencer lorsqu'elle atteint le maximum. NOCYCLE interdit à la séquence de générer des valeurs lorsqu'elle atteint ce seuil.

CACHE permet d'optimiser l'utilisation des séquences en plaçant en mémoire un certain nombre de valeurs générées par la séquence.

Exemple : Créer une séquence Serv_id qui sera utilisée pour les valeurs de la clé primaire de la table service (sans cycle).

Les séquences (5/10)

Ex: `CREATE SEQUENCE Serv_id ...CACHE 100;`

indique que les 100 valeurs successives sont générées en même temps et sont placées dans la mémoire.

Un autre ensemble de valeurs est généré seulement si toutes les valeurs de la mémoire sont utilisées.

***RQ!** Attention, quand la base est arrêtée, le contenu du cache est perdu, ce qui peut provoquer une discontinuité dans les valeurs de la séquence.*

Les séquences (6/10)

Interrogation d'une Séquence

L'interrogation d'une séquence se fait par l'utilisation des pseudo colonnes : **CURRVAL** et **NEXTVAL**.

- **NEXTVAL** : *incrémente la séquence et retourne la nouvelle valeur. Si c'est le premier appel, alors elle initialise la séquence.*
- **CURRVAL** : *retourne la valeur courante de la séquence. Elle ne peut être appelée que si NEXTVAL a été appelée au moins une fois (pour l'initialisation).*

Les séquences (7/10)

Utilisation

```
INSERT INTO nom_table VALUES  
(nom_sequence.NEXTVAL, listes_colonnes ...);  
  
SELECT nom_sequence.CURRVAL FROM SYS.dual;
```

Les séquences (8/10)

Modification d'une séquence:

Il est possible de modifier tous les paramètres d'une séquence (à part le paramètre START WITH).

```
ALTER SEQUENCE nom_séquence  
[INCREMENT BY <valeur-pas>]  
[MAXVALUE <valeur-max> | NOMAXVALUE]  
[CYCLE | NOCYCLE]  
[CACHE <nb-valeurs> | NOCACHE];
```

Remarques:

- ➔ La modification des paramètres d'une séquence n'affecte pas sa valeur courante (elle reste inchangée).
- ➔ La séquence doit être supprimée puis recréée pour la faire repartir d'un numéro différent.

Les séquences (9/10)

Suppression d'une séquence :

```
DROP SEQUENCE nom_séquence;
```

Consultation d'une séquence

Toutes les informations concernant les séquences créées sont disponibles dans la vue `user_sequences` du dictionnaire des données:

```
SELECT sequence_name, min_value, max_value increment_by,  
last_number  
FROM user_sequences;
```

`Last_number` : dernier numéro généré.

Les séquences (10/10)

Remarques

Etant donnée qu'une séquence peut être interrogée à tout moment par tout utilisateur ayant les droits suffisants, il ne faut pas considérer les séquences comme un moyen de générer une suite de nombre continue

Dans le cas de l'alimentation d'une clé primaire, si un enregistrement a été inséré puis la transaction a été annulée, alors la séquence ne revient pas en arrière, et lors de l'insertion suivante, une nouvelle valeur est générée!

Une séquence fournit un moyen d'obtenir des valeurs uniques, mais pas forcément continues.

Exercice

1. Créer une séquence `Serv_id` qui sera utilisée pour les valeurs de la clé primaire de la table `service` (sans cycle).
2. Insérer des lignes dans la table `service` en utilisant la séquence `serv_id` pour définir la valeur de la clé primaire (le contenu de la table n'est pas important).
3. Consulter la séquence pour connaître sa valeur courante.
4. Modifier le pas d'incrément à 20 au lieu de 10 dans `Serv_Id`.
5. Puis insérer un autre service.
6. Consulter la séquence.